Enfoque modular y diseño Top-Down: desarrollo de una calculadora de área con representación de diana en consola

Sofía Mojica, Gabriel Zacarias, Mateo Roberts y Leonel De Armas

E.E.S.T. N.° 1 “Eduardo Ader”, Vicente López

Laboratorio de Programación (LPR) — 5.° 3.ª A-B

Prof. Mansilla Muñoz, York E., y Prof. Ganduglia, Yamil

23 de octubre de 2026

Introducción

La presente actividad forma parte del proyecto integrador anual de la materia Laboratorio de Programación de 5.° año y tiene como propósito aplicar los principios de la programación estructurada al desarrollo de una herramienta de cálculo geométrico. El problema planteado —obtener el área de un círculo a partir de la medida de su radio— es sencillo desde el punto de vista matemático, pero exige tomar decisiones de diseño que resultan representativas de cualquier desarrollo de software de mayor escala: cómo dividir el problema en partes manejables, cómo definir los contratos entre esas partes y cómo proteger al sistema frente a datos que carecen de sentido físico.

Para abordarlo se adoptó el enfoque Top-Down, una estrategia de descomposición que parte de una formulación general del problema y la subdivide de manera sucesiva en subproblemas de menor complejidad, hasta obtener módulos que pueden implementarse y verificarse de forma independiente (Pressman & Maxim, 2021; Sommerville, 2016). En C++, cada uno de esos módulos se materializa en una función: una unidad de código con un nombre, una lista de parámetros y un valor de retorno que puede invocarse desde distintos puntos del programa (Stroustrup, 2014). La combinación de ambas ideas —descomposición y encapsulamiento en funciones— constituye el núcleo del trabajo.

El objetivo general de la actividad es diseñar, implementar y documentar una calculadora modular de área en C++ que integre la validación de los datos de entrada y una representación gráfica en consola del resultado. Los objetivos específicos son los siguientes:

- analizar los conceptos de diseño Top-Down, modularización y prototipo de función en el contexto de la programación estructurada;
- descomponer el problema en módulos con responsabilidades, entradas y salidas claramente delimitadas;
- implementar el módulo de cálculo y el filtro de consistencia, aplicando el calificador const a los valores que no deben modificarse;

- verificar el comportamiento del programa mediante casos de prueba que incluyan valores límite y entradas inválidas;
- documentar la estructura del repositorio, la distribución de roles y los resultados obtenidos.

El informe se organiza en cuatro apartados. El desarrollo presenta, en primer lugar, el marco conceptual que sustenta la solución; luego describe la descomposición modular y su implementación en C++; a continuación analiza la validación de entradas y los resultados de la ejecución; y, por último, documenta la organización del repositorio y las respuestas al cuestionario técnico. Después se exponen las conclusiones y las referencias, y en el anexo se reproduce el código fuente completo.

Desarrollo

Programación estructurada y enfoque Top-Down

La programación estructurada organiza el código a partir de tres estructuras de control básicas —secuencia, selección e iteración— y de la descomposición del programa en funciones, de modo que cada bloque tenga una única responsabilidad y un punto de entrada y de salida bien definidos (Kernighan & Ritchie, 1988). Esta forma de trabajo se apoya en el principio de abstracción: quien invoca una función necesita conocer qué hace y qué datos requiere, pero no cómo está implementada por dentro. Gracias a ello, el programa puede razonarse por niveles: primero el flujo general y después el detalle de cada módulo.

El diseño Top-Down lleva ese principio al proceso de resolución. Consiste en formular el problema en su nivel más general y dividirlo en subproblemas cada vez más pequeños, hasta llegar a operaciones que pueden resolverse con pocas instrucciones (Sommerville, 2016). Cada división produce un módulo, y cada módulo se comunica con los demás a través de una interfaz explícita. La calidad de esa interfaz —su claridad, su estabilidad y la precisión con la que documenta los datos que recibe y devuelve— determina en gran medida el éxito del diseño (Pressman & Maxim, 2021).

La modularización ofrece ventajas concretas: reduce la complejidad cognitiva de cada fragmento, facilita la depuración localizada, permite reutilizar el código en otros programas y hace posible que varias personas trabajen en paralelo sobre partes distintas del sistema (McConnell, 2004). También impone riesgos, entre ellos la definición de interfaces ambiguas y la duplicación de lógica cuando los límites entre módulos no se establecen con criterio. Para evitarlos, el equipo adoptó dos reglas: cada función resuelve una sola tarea y ninguno de los módulos accede directamente a las variables de otro. El vocabulario técnico empleado en el informe se sintetiza en la Tabla 1.

Tabla 1

Glosario técnico de la solución modular

Concepto	Definición	Aplicación en el proyecto
Diseño Top-Down	Estrategia analítica que subdivide un problema complejo en subproblemas de menor nivel hasta obtener operaciones simples.	Organiza el programa en tres niveles: coordinación, operaciones y fórmula.
Modularización	Fragmentación del código en unidades autónomas con una responsabilidad delimitada.	Separa entrada, validación, cálculo y salida en funciones independientes.
Prototipo de función	Declaración anticipada de la firma de un módulo: tipo de retorno, nombre y parámetros.	Permite que main() invoque calcularAreaCirculo() antes de su definición.
Retorno (return)	Mecanismo que transfiere el resultado de un subprograma al contexto que lo invocó.	Devuelve el área calculada y, en main(), el código de estado del proceso.
Constante (const)	Calificador que impide la reasignación de un valor durante la ejecución.	Protege el valor de π utilizado en la fórmula.
Filtro de consistencia	Validación que rechaza datos fuera del dominio físico o lógico del problema.	Descarta radios menores o iguales a cero antes del cálculo.

Nota. Elaboración propia a partir de Pressman y Maxim (2021), Sommerville (2016) y Stroustrup (2014).

Descomposición modular del problema

El problema se organizó en tres niveles de abstracción. En el nivel 0 se ubica la función main(), que actúa como coordinadora: no calcula nada por sí misma, sino que ordena la ejecución de los demás módulos. En el nivel 1 se encuentran las operaciones de

entrada, validación, cálculo y salida. En el nivel 2 se localiza la fórmula geométrica propiamente dicha, aislada dentro de calcularAreaCirculo(). Esta jerarquía permite que cualquier cambio en la fórmula no afecte la lectura de datos ni la presentación del resultado. La Tabla 2 detalla cada módulo y la Figura 1 representa el flujo completo de la ejecución.

Tabla 2

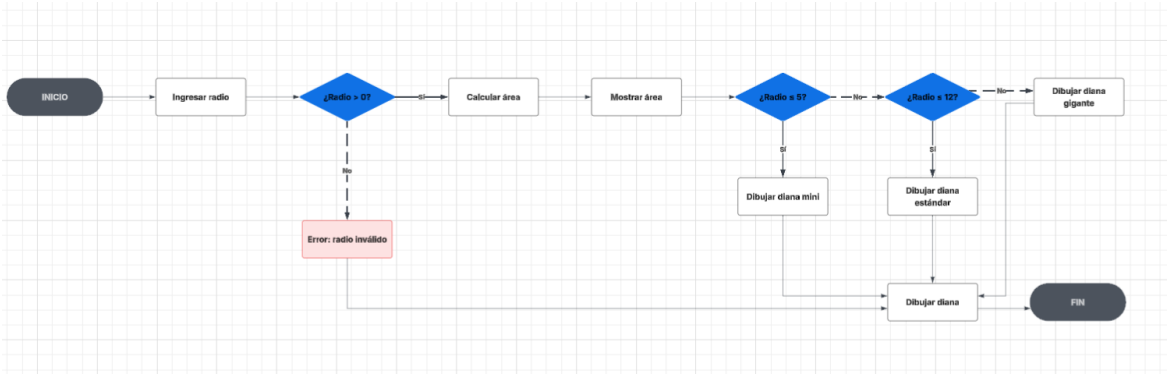
Descomposición Top-Down: módulos, responsabilidades y datos

Nivel	Módulo	Responsabilidad	Entrada	Salida
0	main()	Coordinar el flujo general del programa.	—	Código de estado
1	Entrada de datos	Leer el radio ingresado por el usuario.	Texto de consola	radioEstudiante
1	Filtro de consistencia	Rechazar radios no positivos.	radioEstudiante	Error o continuación
1	calcularAreaCirculo()	Calcular el área a partir del radio.	radio (double)	area (double)
2	Fórmula geométrica	Resolver la expresión π · r².	radio y π	Área
1	Salida de datos	Presentar el resultado y la diana ASCII.	area y radio	Texto en consola

Nota. Elaboración propia. Los niveles indican la profundidad de la descomposición: el nivel 0 corresponde al programa principal y el nivel 2 al detalle de la fórmula.

Figura 1

Diagrama de flujo del algoritmo modular



Nota. Elaboración propia. El diagrama representa la secuencia de entrada, validación, cálculo, salida y representación de la diana.

La descomposición adoptada respeta el criterio de ocultamiento de información: el módulo de cálculo no imprime, no lee datos y no valida; solo transforma un valor de entrada

en un valor de salida. Esta pureza funcional —ausencia de efectos secundarios— es la que permite probarlo de manera aislada y, eventualmente, reutilizarlo en otra aplicación (McConnell, 2004). El filtro de consistencia, en cambio, concentra la política de aceptación de datos y constituye el único punto del programa donde se decide si una entrada es admisible.

Implementación del módulo de cálculo

La firma del módulo se declara antes de main() mediante un prototipo: double calcularAreaCirculo(double radio);. El prototipo informa al compilador el tipo de retorno, el nombre de la función y la cantidad y el tipo de sus argumentos, de modo que las llamadas puedan verificarse durante la compilación (ISO/IEC, 2020; cppreference.com, 2024a). Sin esa declaración anticipada, el compilador no dispone de la información necesaria para validar la llamada y el programa no se compila.

La definición de la función declara la constante PI con el calificador const, de modo que el valor no pueda reasignarse una vez inicializado. Esta práctica, conocida como corrección de constantes, permite que el compilador detecte asignaciones accidentales y habilita optimizaciones sobre el valor (cppreference.com, 2024b; Stroustrup, 2014). El parámetro r se recibe por valor, es decir, como una copia del argumento: cualquier modificación interna no altera la variable radioEstudiante del programa principal.

El cálculo se resuelve con la expresión area = PI * r * r; y el resultado se devuelve con return. El tipo double ofrece una precisión de aproximadamente quince cifras significativas, suficiente para las magnitudes del problema: el truncamiento de π en la décima cifra decimal introduce un error relativo de aproximadamente 0,00000000003, despreciable frente a la precisión de cualquier medición escolar. La Figura 2 reproduce los fragmentos esenciales de la implementación; el código completo se incluye en el Anexo A.

Figura 2

Fragmentos esenciales de la implementación modular

```
// 1. Prototipo: la firma del modulo se declara antes de main()
double calcularAreaCirculo(double radio);

int main() {
    double radioEstudiante;
    // ... entrada de datos por consola ...

    // Filtro de consistencia: validacion del dato geometrico
    if (radioEstudiante <= 0) {
        cout << "[ERROR] El radio debe ser un valor positivo y mayor a cero." << endl;
        return 1;   // salida controlada con codigo de error
    }

    double areaFinal = calcularAreaCirculo(radioEstudiante);
    // ... salida de datos y representacion de la diana ...
    return 0;
}

// 2. Definicion: formula aislada, sin efectos secundarios
double calcularAreaCirculo(double r) {
    const double PI = 3.1415926535;
    double area = PI * r * r;
    return area;
}
```

Nota. Elaboración propia. Los comentarios con puntos suspensivos indican fragmentos omitidos; el listado completo se presenta en el Anexo A.

La traducción del código fuente a un ejecutable se realizó con el compilador MinGW-w64, invocado desde la terminal integrada del entorno de desarrollo. El comando g++ src/main.cpp -o src/calculoarea.exe cumple dos funciones: verifica la sintaxis y la coherencia de tipos de cada unidad de traducción y, en la etapa de enlace, resuelve las referencias entre la llamada y la definición del módulo. La ejecución posterior se realiza con .\src\calculoarea.exe.

Validación de entradas: el filtro de consistencia

Un programa correcto no es únicamente aquel que produce el resultado esperado con datos válidos; también debe comportarse de manera predecible cuando recibe datos que no pertenecen al dominio del problema (Myers et al., 2012). En el caso del área, el dominio está formado por los números reales positivos: un radio de 0 cm no define una superficie y un radio negativo carece de sentido en el espacio euclidiano. Si el filtro no existiera, el programa devolvería un valor positivo para una entrada negativa —porque el

cuadrado elimina el signo—, es decir, ocultaría el error y entregaría un resultado geométricamente inválido.

El filtro implementado evalúa la condición radioEstudiante <= 0 e interrumpe la ejecución con un mensaje explícito y un código de salida distinto de cero. La decisión de interrumpir en lugar de solicitar nuevamente el dato responde a una consigna de la actividad, aunque en una versión de producción sería preferible un bucle de relectura que permitiera corregir la entrada sin reiniciar el proceso. El retorno 1 permite que un script o un sistema operativo detecte que el programa terminó por un error, convención habitual en las herramientas de línea de comandos.

La Tabla 3 resume los casos de prueba aplicados, construidos a partir de los valores límite del dominio y de las tres escalas de representación (Myers et al., 2012).

Tabla 3

Casos de prueba y resultados obtenidos

Caso	Entrada (cm)	Resultado esperado	Salida del sistema	Estado
1	5	78,5398 cm²	Área y diana compacta	Correcto
2	12	452,3893 cm²	Área y diana estándar	Correcto
3	25	1963,4954 cm²	Área y diana gigante	Correcto
4	0	Rechazo de la entrada	Mensaje de error; código 1	Correcto
5	−5	Rechazo de la entrada	Mensaje de error; código 1	Correcto
6	“abc”	Rechazo de la entrada	Lectura fallida sin aviso	Limitación

Nota. Elaboración propia. Las áreas se expresan con cuatro decimales y se obtuvieron con el valor de π declarado en el programa. El caso 6 no estaba contemplado en el diseño original y se analiza en las conclusiones.

Los casos 1 a 3 verifican la fórmula y la selección de la escala de la diana; los casos 4 y 5 confirman que el filtro intercepta los valores fuera del dominio antes de ejecutar el cálculo. El caso 6 pone de manifiesto una limitación: si el usuario ingresa caracteres no numéricos, cin entra en estado de fallo, la variable conserva un valor indefinido y la

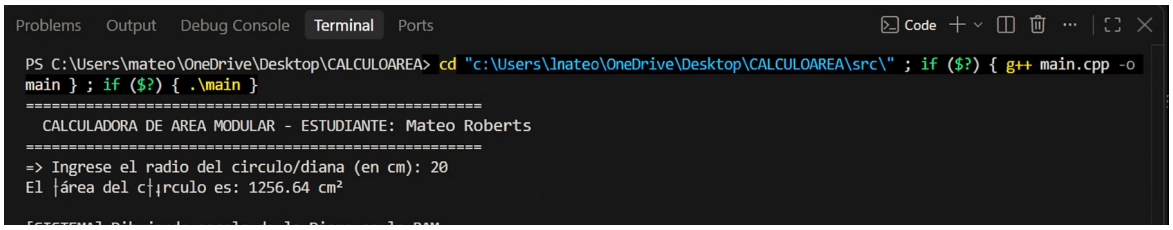
comparación con cero no alcanza para detectar el problema. Esta observación se retoma en las conclusiones como línea de mejora.

Resultados de la ejecución

La Figura 3 muestra la compilación y la ejecución del programa en la terminal. La compilación finaliza sin errores ni advertencias, y la ejecución presenta el mensaje de bienvenida, solicita el radio, informa el área calculada y dibuja la diana correspondiente a la escala del valor ingresado.

Figura 3

Compilación y ejecución del programa en la terminal



Nota. Elaboración propia. La captura corresponde a una ejecución completa con entrada válida y salida por consola.

La diana se selecciona por intervalos: hasta 5 cm se dibuja la versión compacta, hasta 12 cm la estándar y por encima de ese valor la gigante. Conviene señalar que la diana es un recurso de gamificación y no una representación métrica: las tres plantillas son fijas y no guardan proporción con el radio. Su función es reforzar la lectura del resultado y no sustituir la magnitud numérica, que es la que conserva validez geométrica. La Figura 4 reproduce las tres variantes de la salida.

Figura 4

Escalas de la diana de tiro generadas en la consola

```
radio <= 5 cm                5 < radio <= 12 cm              radio > 12 cm

    .---.                        .---.                          .---.
   /  X  \                     / .---. \                      / .---. \
   \  *  /                    | /  O  \ |                     | / .-. \ |
    '---'                     | \  *  / |                     | |  X  | |
                              \ '---' /                      | \ '-' / |
  [DIANA MINI]                 '---'                         \ '---' /
                               [DIANA ESTANDAR]                '---'
                                                            [DIANA GIGANTE]
```

Nota. Elaboración propia. Las tres plantillas se seleccionan mediante una estructura condicional múltiple y no representan la magnitud del área calculada.

Organización del repositorio y trabajo colaborativo

El repositorio se estructuró para separar el código fuente, la documentación y las evidencias de ejecución, de modo que cualquier integrante pudiera localizar un archivo sin recorrer el proyecto completo. La Tabla 4 describe la organización de carpetas y archivos.

Tabla 4

Estructura del repositorio del proyecto

Ruta	Contenido
/	Raíz del proyecto calculoarea.
.gitignore	Exclusiones de Git para configuraciones locales (.vscode).
LICENSE	Licencia MIT del proyecto escolar.
README.md	Presentación técnica y comandos de compilación y ejecución.
docs/	Documentación y entregables en formato PDF.
src/main.cpp	Código fuente modular en C++.
capturas/	Evidencias de compilación y diagrama de flujo.

Nota. Elaboración propia a partir de la estructura publicada en el repositorio.

El control de versiones se gestionó con Git y GitHub. El archivo README.md concentra la información de instalación y ejecución; la licencia MIT habilita el uso escolar del código; y la carpeta docs/ reúne los entregables institucionales. La distribución de roles se presenta en la Tabla 5: cada integrante asumió una responsabilidad principal, aunque las decisiones de diseño se tomaron en conjunto.

Tabla 5

Distribución de roles y tareas del equipo

Integrante	Rol	Tareas asignadas
Sofía Mojica	Análisis de algoritmos y pruebas	Diseño e implementación del filtro de consistencia y verificación de casos límite.
Gabriel Zacarias	Ingeniería de software	Desarrollo del módulo matemático, integración del prototipo y revisión de la fórmula.
Mateo Roberts	Arquitectura y control de versiones	Estructuración del repositorio, renderizado de la diana ASCII y gestión de Git y GitHub.
Leonel De Armas	Documentación técnica	Diagrama de flujo, respuestas al cuestionario y redacción del informe en APA 7.

Nota. Elaboración propia.

El repositorio del proyecto se encuentra disponible en el siguiente enlace: https://github.com/matuteXEN6/ACTIVIDAD-3-MANIPULACI-N-DE-CADENAS-Y-DE-PUNTEROS-.git (De Armas et al., 2026).

El trabajo colaborativo se apoyó en dos acuerdos. El primero fue la integración temprana: cada avance se subía a una rama propia y se revisaba antes de incorporarse a la principal, con el objetivo de evitar conflictos acumulados. El segundo fue la verificación cruzada: quien escribía una función no era quien la probaba, de modo que las pruebas se diseñaran a partir de la interfaz y no del código, tal como recomienda la literatura sobre pruebas de caja negra (Myers et al., 2012).

Análisis del cuestionario técnico

Función del prototipo en la compilación

El compilador de C++ procesa el archivo de arriba hacia abajo. Si se suprime el prototipo, cuando alcanza la llamada dentro de main() todavía no conoce la firma de calcularAreaCirculo(): no sabe cuántos bytes ocupa el parámetro, qué tipo recibe ni qué valor devuelve. La compilación se interrumpe entonces con un error del tipo “identifier not found” o “was not declared in this scope”. El prototipo funciona como una declaración anticipada que registra la firma en la tabla de símbolos y permite verificar la cantidad y el

tipo de los argumentos en cada invocación (cppreference.com, 2024a). En un diseño modular cumple, además, una función documental: expresa el contrato público del módulo antes de mostrar su implementación.

Consecuencias de admitir radios no positivos

Desde el punto de vista geométrico, un radio de 0 cm define un punto sin superficie y un radio negativo constituye una incongruencia espacial inexistente en el espacio euclidiano. Desde el punto de vista del programa, el problema es más grave de lo que parece: como la fórmula eleva el radio al cuadrado, una entrada de −5 cm produce el mismo resultado que una de 5 cm, de modo que el error queda oculto y el usuario recibe un valor que parece correcto. El filtro preventivo evita esa situación, impide consumir ciclos de reloj en un cálculo sin sentido y garantiza que toda salida del sistema provenga de un dato físicamente admisible (Myers et al., 2012).

Protección de constantes en memoria

En sistemas críticos, como los utilizados en aviación o en medicina, la alteración involuntaria de una variable de control —por un desbordamiento de memoria o por un error de asignación lógica— puede modificar cálculos de trayectoria o dosis de fármacos con consecuencias graves. El calificador const instruye al compilador para rechazar cualquier intento de reasignación: el valor se inicializa una sola vez y se trata como de solo lectura durante el resto del proceso (cppreference.com, 2024b). Si un error del programa intenta reescribirlo, la construcción del ejecutable falla de inmediato y el defecto se detecta antes de la puesta en producción, que es el momento en que resulta más económico corregirlo (Pressman & Maxim, 2021). En la calculadora, esta protección alcanza al valor de π, cuya estabilidad es condición necesaria para que el resultado sea reproducible.

Conclusión

El desarrollo de la calculadora permitió comprobar que el enfoque Top-Down no es un formalismo metodológico, sino una herramienta con efectos concretos sobre la calidad del código. La separación entre entrada, validación, cálculo y salida produjo un programa en el que cada función puede explicarse en una oración y probarse de manera independiente.

Esa propiedad, que la literatura asocia con la cohesión y el bajo acoplamiento (Pressman & Maxim, 2021), se verificó en la práctica: al modificar la fórmula o el mensaje de error, ninguna otra parte del sistema dejó de funcionar.

Los resultados obtenidos fueron consistentes con lo esperado. La compilación se completó sin errores ni advertencias; los casos válidos arrojaron las áreas calculadas con cuatro decimales y la diana correspondiente a cada escala; y los casos límite —radio nulo y radio negativo— fueron rechazados con un mensaje explícito y un código de salida distinto de cero. La incorporación de const al valor de π y el uso de un prototipo para declarar la firma del módulo completaron un diseño en el que el compilador actúa como primera instancia de control.

El trabajo también dejó limitaciones visibles. La entrada no numérica provoca el estado de fallo de cin y no recibe un tratamiento específico; la diana ASCII es una representación cualitativa y no métrica; y todo el programa se concentra en un único archivo, lo que resulta adecuado para la escala de la actividad pero no para un sistema de mayor tamaño. Reconocer estas fronteras es parte del aprendizaje: un informe técnico debe distinguir lo que el sistema garantiza de lo que todavía no resuelve.

Como proyección, el equipo se propone validar la entrada con cin.fail() y un bucle de relectura, separar las declaraciones y las definiciones en archivos de cabecera, incorporar pruebas automatizadas para los valores límite y parametrizar las escalas de la diana. Ninguna de estas mejoras altera la arquitectura general: todas consisten en agregar módulos o reforzar los existentes, lo que confirma que la descomposición elegida admite crecimiento sin reescritura. El enfoque modular, en definitiva, se demostró como una inversión de diseño que se recupera en cada cambio posterior.

Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.ª ed.). https://doi.org/10.1037/0000165-000

cppreference.com. (2024a). *Functions*. https://en.cppreference.com/w/cpp/language/functions

cppreference.com. (2024b). *cv (const and volatile) type qualifiers*. https://en.cppreference.com/w/cpp/language/cv

De Armas, L., Mojica, S., Roberts, M., & Zacarias, G. (2026). *Actividad 3: Enfoque modular y diseño Top-Down* [Código fuente]. GitHub. https://github.com/matuteXEN6/ACTIVIDAD-3-MANIPULACI-N-DE-CADENAS-Y-DE-PUNTEROS-.git

International Organization for Standardization. (2020). *ISO/IEC 14882:2020. Programming languages — C++*. https://www.iso.org/standard/79358.html

Kernighan, B. W., & Ritchie, D. M. (1988). *The C programming language* (2.ª ed.). Prentice Hall.

McConnell, S. (2004). *Code complete* (2.ª ed.). Microsoft Press.

Myers, G. J., Sandler, C., & Badgett, T. (2012). *The art of software testing* (3.ª ed.). John Wiley & Sons.

Pressman, R. S., & Maxim, B. R. (2021). *Ingeniería del software: Un enfoque práctico* (9.ª ed.). McGraw-Hill.

Sommerville, I. (2016). *Software engineering* (10.ª ed.). Pearson.

Stroustrup, B. (2014). *Programming: Principles and practice using C++* (2.ª ed.). Addison-Wesley.

Anexo A

Código fuente del programa modular en C++

Se reproduce a continuación el contenido íntegro de src/main.cpp, correspondiente a la versión entregada en el repositorio del proyecto.

```
/*
   ================================================================
   Materia: Laboratorio de Programación (LPR)
   E.E.S.T. N.º 1 "Eduardo Ader" - Vicente López
   Curso: 5.º año
   Profesor: Prof. Mansilla Muñoz, York E.
   Archivo: main.cpp
   Objetivo: calcular el área de un círculo mediante funciones modulares
             y representar una diana de tiro en ASCII según la escala
             del radio ingresado.
   ================================================================
*/

#include <iostream>

using namespace std;

// ================================================================
// 1. PROTOTIPO DE LA FUNCIÓN (firma del módulo)
// Se informa al compilador la existencia del módulo de cálculo
// antes de main().
// ================================================================
double calcularAreaCirculo(double radio);

int main() {
    double radioEstudiante;

    // ============================================================
    // REGLA OBLIGATORIA (EVITA EL PLAGIO):
    // Modificar la salida de pantalla agregando nombre y apellido.
    // ============================================================
    cout << "======================================================" << endl;
    cout << "  CALCULADORA DE AREA MODULAR - ESTUDIANTE: Leonel De Armas" << endl;
    cout << "======================================================" << endl;

    cout << "=> Ingrese el radio del circulo/diana (en cm): ";
    cin >> radioEstudiante;

    // FILTRO DE CONSISTENCIA (validación del dato geométrico)
    if (radioEstudiante <= 0) {
        cout << "[ERROR] El radio debe ser un valor positivo y mayor a cero." << endl;
        return 1; // Salida controlada indicando error en consola
    }

    // ============================================================
    // 2. LLAMADA / INVOCACIÓN DE LA FUNCIÓN
    // Se convoca al módulo calcularAreaCirculo() pasándole
    // radioEstudiante y se guarda el resultado en areaFinal.
    // ============================================================
    double areaFinal = calcularAreaCirculo(radioEstudiante);

    // ============================================================
    // 3. SALIDA DE DATOS
    // Se muestra en pantalla el área calculada.
    // ============================================================
    cout << "El area del circulo es: " << areaFinal << " cm2" << endl;

    // ============================================================
    // BONUS GAMIFICACIÓN: RENDER DE LA DIANA EN CONSOLA
    // Según el radio ingresado, la consola simula un blanco de tiro
    // correspondiente al tamaño calculado.
    // ============================================================
    cout << "\n[SISTEMA] Dibujando escala de la Diana en la RAM..." << endl;
    if (radioEstudiante <= 5.0) {
        cout << "       .---.       " << endl;
        cout << "      /  X  \\      -> [DIANA MINI / COMPACTA]" << endl;
        cout << "      \\  *  /      " << endl;
        cout << "       '---'       " << endl;
    } else if (radioEstudiante <= 12.0) {
        cout << "       .---.       " << endl;
        cout << "     / .---. \\     " << endl;
        cout << "    | /  O  \\ |    -> [DIANA ESTÁNDAR DE TIRO]" << endl;
        cout << "    | \\  *  / |    " << endl;
        cout << "     \\ '---' /     " << endl;
        cout << "       '---'       " << endl;
    } else {
        cout << "       .---.       " << endl;
        cout << "     / .---. \\     " << endl;
        cout << "    | / .-. \\ |    " << endl;
        cout << "    | |  X  | |    -> [DIANA GIGANTE DE COBERTURA]" << endl;
        cout << "    | \\ '-' / |    " << endl;
        cout << "     \\ '---' /     " << endl;
        cout << "       '---'       " << endl;
```

```
    }

    cout << "=================================================" << endl;
    return 0; // Finalización exitosa (código 0)
}

// ====================================================================
// 4. DEFINICIÓN / DESARROLLO DE LA FUNCIÓN
// Se implementa la fórmula de la geometría utilizando variables
// locales y la constante pi almacenada en memoria de solo lectura.
// ====================================================================
double calcularAreaCirculo(double r) {
    const double PI = 3.1415926535;

    // Fórmula del área: pi * radio^2
    double area = PI * r * r;

    return area; // Se retorna el resultado calculado
}
```

Nota. Elaboración propia. Se normalizó la codificación de caracteres a UTF-8 y la alineación de comentarios y del arte ASCII; la lógica del programa no fue modificada. En la consola de Windows, la salida de caracteres acentuados depende de la página de códigos activa (por ejemplo, chcp 65001).